编译器设计专题实验课2026

2026.05

一、实验五：语义分析框架

- **实验五：SLR引导的语义分析框架实现**
- 在SLR(1)语法分析的基础上，集成语义动作，构建抽象语法树（AST）并维护符号表，完成类型检查和初步语义验证。
- **文法扩展-在产生式中嵌入语义动作；属性计算-实现属性值的传递（如表达式类型、变量地址等）；符号表管理-**支持变量声明、作用域嵌套和类型查询；错误检测-报告重复声明、类型不匹配等语义错误。

一、实验五：语义分析框架

- 实验五：SLR引导的语义分析框架实现
- 输入：-SLR(1)分析表（实验四输出）嵌入语义动作的文法规则（文件或硬编码）
- - Token流（实验二输出）
- 输出：- 抽象语法树（AST）的可视化表示，符号表（变量名、类型、作用域），语义错误报告（如类型错误）
- 在SLR(1)语法分析的基础上，集成语义动作，构建抽象语法树（AST）并维护符号表，完成类型检查和初步语义验证。

一、实验五：语义分析框架

产生式	语义规则
S→id=E	S.code = E.code ‖gen(top.get(id.lexeme)'='E.addr)
$E \rightarrow E_1+E_2$	E.addr = new Temp(); $E.code = E_1.code \| E_2.code$ ‖gen(E.addr'='E_1.addr '+'E_2.addr)
$E \rightarrow E_1*E_2$	E.addr = new Temp(); $E.code = E_1.code \| E_2.code$ ‖gen(E.addr':='E_1.addr '*'E_2.addr)
$E \rightarrow (E_1)$	E.addr = E_1.addr; E.code = E_1.code
E→id	E.addr = top.get(id.lexeme); E.code = ''

一、实验五：语义分析框架

- 实验六、七八的安排和输入输出

实验阶段	输入	输出	核心任务
实验六	AST、符号表	中间代码（三地址码、四元式或LLVM IR）	语法树翻译为中间表示
实验七	中间代码、符号表	内存地址映射文件	内存地址分配与代码序列化
实验八	中间代码、内存映像、符号表	汇编/可执行代码	目标代码生成与优化

第七次课程开始验收，同学们可以参考思源学堂的示例代码开始准备。验收阶段从源程序阶段到中间代码或者目标代码阶段。

二、实验步骤和结果

- 实验步骤：
- 文法扩展：为每个产生式添加语义动作（例如生成AST节点或更新符号表）；
- 属性栈设计：在语法分析栈旁维护属性栈，保存每个符号的语义值（如AST节点指针、类型）
- 语义动作执行：规约时调用对应的语义动作函数，操作属性栈和符号表。
- 符号表实现：支持作用域栈结构，处理变量声明和引用。
- 错误处理：在语义动作中检测错误（如未声明变量）并记录。

二、实验步骤和结果

```
root@dsw-544537-86b6854df5-k9849:/mnt/workspace# g++ -std=c++17 -I /usr/include/nlohmann/ semantic.cpp -o sema
root@dsw-544537-86b6854df5-k9849:/mnt/workspace# ./semantic
请输入SLR分析表文件: SLR1.txt
请输入Token流文件: token.txt
terminate called after throwing an instance of 'nlohmann::detail::parse_error'
  what(): [json.exception.parse_error.101] parse error at line 2, column 37: syntax error while parsing objec
iteral; last read: '"s2"<U+FFFD>; expected '}'
已中止
```

三、实验六：中间代码生成

- 实验六：中间代码生成
- 输入抽象语法树信息或其他，token，源程序等等。
- 输出中间代码表示，三元式、四元式、三地址方式均可，以下输出均可

```
input x;
if 0 < x then
        fact = 1
else fact=2;
repeat
        fact = fact * x;
        x = x – 1
until x == 0;
print fact
```

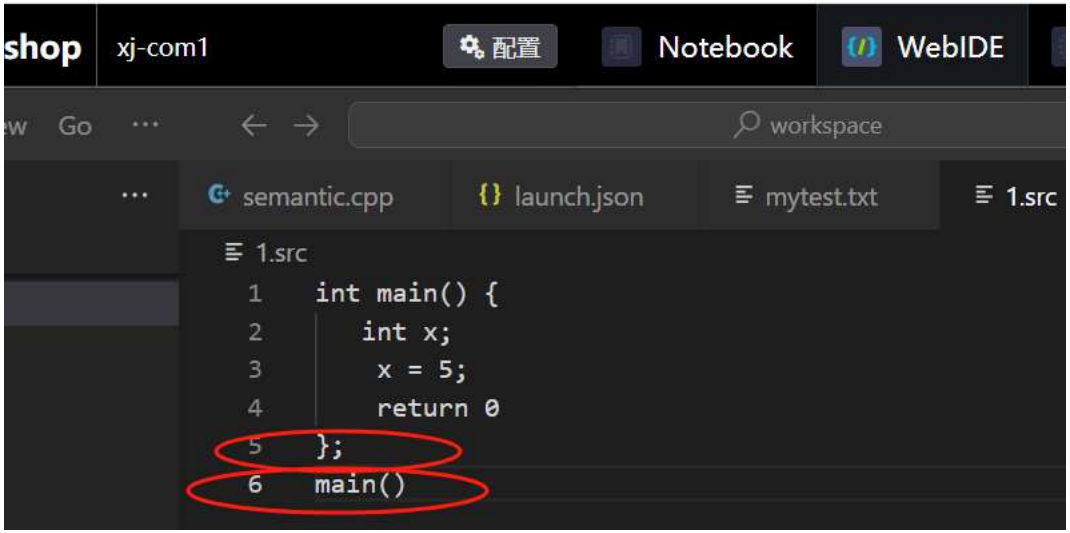
```
INPUT x
t1 = 0
IF t1 < x THEN l1 ELSE l2
LABEL l1
fact = 1
GOTO l3
LABEL l2
fact = 2
LABEL l3
fact = fact * x
x = x – 1
IF x == 0 THEN l4 ELSE l3
LABEL l4
PRINT fact
```

例如 X = a*b+c/d 的四元式序列：

1. `(*, a, b, T1)`
2. `(/, c, d, T2)`
3. `(+, T1, T2, T3)`
4. `(=, T3, -, X)`

三、实验六：中间代码生成

- 实验六：中间代码生成
- 示例代码1



- 示例代码库已经上传，根据同学的修正，目前正确代码大多数，错误代码非常少，集中在词法分析阶段，能够减轻同学在出错处理阶段精力，主要用于分析文法。

四、示例代码库和文法的解释

- 词法规则文件

```
d ~ (ID, name)
ID = [a-z][a-z0-9]*
i ~ (INT, value)
NUM = [+-]?[0-9]+
f ~ (FLO, value)
FLO = [+-]?[0-9]*. [0-9]+ | [+-]?[0-9]+. [0-9]*
+ ~ (ADD, -)
ADD = {+}
* ~ (MUL, -)
MUL = {*}
r ~ (ROP, name)
ROP = {<, <=, ==}
\= ~ (ASG, -)
ASG = {\=}
\( ~ (LPA, -)
LPA = {\(}
\) ~ (RPA, -)
RPA = {\)}
[ ~ (LBK, -)
LBK = {[}
] ~ (RBK, -)
RBK = {]}
\{ ~ (LBR, -)
LBR = {\{}
\} ~ (RBR, -)
RBR = {\}}
\, ~ (CMA, -)
CMA = {\,}
\; ~ (SCO, -)
SCO = {\;}
int ~ (INT, -)
float ~ (FLOAT, -)
if ~ (IF, -)
else ~ (ELSE, -)
while ~ (WHILE, -)
return ~ (RETURN, -)
input ~ (INPUT, -)
print ~ (PRINT, -)
void ~ (VOID, -)
```

四、示例代码库和文法的解释

- 词法类别和正则（附录A）

ID = L([a-z][a-z0-9]*)\KEY
KEY = {int, float, void, if, else, while, return, input, print}
INT = {int}; FLOAT = {float}; VOID = {void}; IF = {if}; ELSE = {else}
WHILE = {while}; RETURN = {return}; INPUT = {input}; PRINT = {print}
NUM = L([+-]?[0-9]*)
FLO = L([+-]?[0-9]*.[0-9]+|[+-]?[0-9]+.[0-9]*)
OP = AOP ∪ ROP ∪ BOP ∪ {=, +=, ++}
AOP = {+, -, *, /}
ADD = {+}; SUB = {-}; MUL = {*}; DIV = {/}
ROP = {<, <=, ==, >, >=, !=}
BOP = {^, v, |} 或 BOP = {&&, ||, !}
ASG = {=}; AAS = {+=}; AAA = {++}
DLI = {(, [, \{, \}, \, _}
LPA = {(}; RPA = {)}; LBK = {[}; RBK = {]}; LBR = {\{}; RBR = {\}}; CMA = {\ ,},SCO = {;}

```
d:ID; int:KEY; int:INT; i:NUM; f:FLO; o:AOP; +:ADD; r:ROP; <:L(<);
    ^:BOP;      ^:L(^);     &&:BOP;   (:DLL;    ):LPA;
```

四、示例代码库和文法的解释

- 二者区别

两份规则的关系：附录A中的词法是功能更完整、定义更严谨的超集或升级版。
主要区别：
标识符定义：附录A中的词法通过 \KEY 操作排除了关键字，避免了词法分析冲突，而第一个词法规则没有，
运算符完整性：第一个词法规则严重缺失关系运算符（如 >、!=）、赋值运算符和逻辑运算符，无法描述完整程序。附录A中的词法定义了完整的运算符集。
请注意数值定义：规则B的整数定义（[0-9]*）可能匹配空字符串。

四、示例代码库和文法的解释

- 主文法

P → Ď Š

Ď → ε | Ď D ;

D → T d | T d[i] | T d (Ă) {Ď Š}

T → int | float | void

Ă → ε | Ă A ;

A → T d | T d[] | T d(T)

Š → S | Š ; S

S → d = E | d[E] = E | if (B) S | if (B) S else S | while (B) S | return E | {Š} | d(Ř) |

E → i | f | d | d[E] | E + E | E * E | (E) | d(Ř)

B →E r E | E

Ř → ε | Ř R ,

R → E | d[] | d()

- 每个变元都指代一种语法单元:程序P，声明表Ď，声明D，类型T，整数表Ǐ，形参表Ă，形参A，Ř实参表，实参R，语句表Š，语句S，表达式表Ě，表达式E，布尔表达式B。

四、示例代码库和文法的解释

- 写成可配置的CFG，程序的输入文件

```
Prog -> DeclList
DeclList -> DeclList Decl | Decl
Decl -> VarDecl | FunDecl
VarDecl -> Type ID SEMI | Type ID LBRACK INT_NUM RBRACK SEMI | Type ID ASG Expr SEMI
Type -> INT | FLOAT | VOID
FunDecl -> Type ID LPAR ParamList RPAR CompStmt
ParamList -> ParamList COMMA Param | Param | ε
Param -> Type ID | Type ID LBRACK RBRACK
CompStmt -> LBR StmtList RBR
StmtList -> StmtList Stmt | ε
Stmt -> VarDecl | OtherStmt
OtherStmt -> ExprStmt | CompStmt | IfStmt | LoopStmt | RetStmt | PrintStmt
PrintStmt -> PRINT LPAR Expr RPAR SEMI
ExprStmt -> Expr SEMI | SEMI
IfStmt -> IF LPAR Expr RPAR CompStmt | IF LPAR Expr RPAR CompStmt ELSE CompStmt
LoopStmt -> WHILE LPAR Expr RPAR Stmt
RetStmt -> RETURN Expr SEMI | RETURN SEMI
Expr -> ID ASG Expr | ID LBRACK Expr RBRACK ASG Expr | ID LPAR ArgList RPAR | SimpExpr
SimpExpr -> AddExpr REL_OP AddExpr | AddExpr
AddExpr -> AddExpr ADD Term | Term
Term -> Term MUL Fact | Fact
Fact -> ID | ID LBRACK Expr RBRACK | INT_NUM | FLOAT_NUM | LPAR Expr RPAR
ArgList -> ArgList COMMA Expr | Expr | ε
```

四、示例代码库和文法的解释

- 示例代码库提供若干初级程序
- 程序段　　1. $P \rightarrow \check{D}\check{S}$
- $\check{D}$ 是变量的声明语句集. $\check{S}$ 是非变量声明语句集, 包括赋值语句

单字母终结符指代一种词法单元:
标识符d，整数i，实数f，算术运算符o，关系运算符r，单目算术运算符u。
此外的终结符都是与自身对应的一个词法单元，这些词法单元包含了关键字、分隔符、括号、逻辑运算符。每个词法单元都是某个词法类别(种属)的一个实例，采用二元组(〈种属〉,〈个体〉) 表示。

1.src - 记事本

文件(F) 编辑(E) 格式(O) 查看

```
int main() {
  int x;
   x = 5;
  return 0
 };
main()
```

函数声明
变量声明
赋值语句
returrn语句

关于分号的用法

S->语句列表之间用分号，而语句结束无需分号

因此return 0 没有分号，函数声明结束部分有分号

2.src - 记事本

文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

```
void func() {
   int a = 3;
   if (a && 5) {
      a = 10;
   }
};
func()
 // 错误: 文法不支持布尔操作符"&&"
```

5.src - 记事本

文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

```
void test() {
   print a
};
test()
```

四、示例代码库和文法的解释

- **示例代码库和文法的解释**
- 此外值得注意的本文法的实参和形参部分
- $A \rightarrow Td()$
- A 代表函数参数列表的声明. 可以是普通变量参数 $A \rightarrow Td$, 可以是数组参数 $A \rightarrow Td[]$, 也可以是函数参数 $A \rightarrow Td()$

Ă → ε | Ă A;↵ Ř → ε | Ř R,↵

- A 是函数参数列表的声明集. A 代表函数参数列表中某个参数的声明.
- 参数之间用分号隔开,而不是逗号, 即使是最后一个参数也需要使用
- 分号结尾.

```
void func1(int abc; int xyz; int arr[]; int func2();)
```

- R 是函数调用参数集. R 是函数参数列表中某个参数
- 的调用. 参数之间用逗号隔开, 最后一个参数需要逗号
- 例如

```
int add(int a; int b; ) {
    return a + b
};

int calc( ) {
    int sum;
    sum = add(3, 4,);
    return sum
};
calc()
```

四、示例代码库和文法的解释

6.src - 记事本

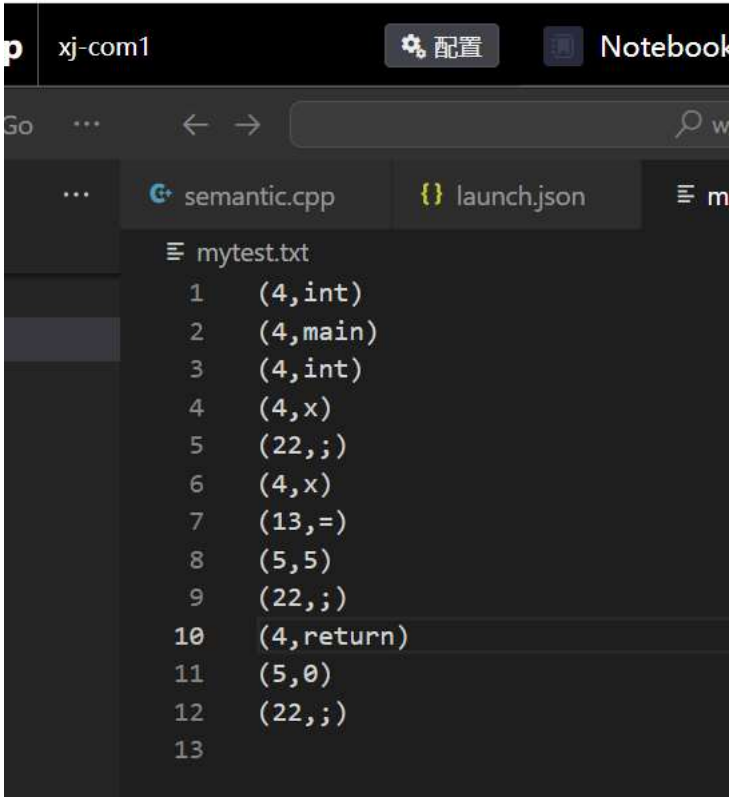
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

```
int main() {
    foo()
};
main()
 // 错误：函数foo未声明
```

- 示例代码库和词法的解释
  - 程序的main被识别为ID，不是关键字
  - 关键字只有int，float，void，if，else，while，return，input和print，
  - 没有 for,除了分号;和逗号，不支持双引号、单引号、\\等其他界符。
  - 没有注释
  - 升级之后的程序支持与或非等操作
  - 支持，{ [(,)] }; ，
  - 验收程序可以支持部分示例文法和示例词法，不需要完全一致。

四、示例代码库和文法的解释

- 词法部分
- 示例程序1.src经过词法以后得到token序列



示例程序10.src

```
root@dsw-544537-57fc4f87f-tn4p9:/mnt/workspace#
root@dsw-544537-57fc4f87f-tn4p9:/mnt/workspace# ./scanner
 please inout source file: 10.src
root@dsw-544537-57fc4f87f-tn4p9:/mnt/workspace#
```

10.src

```
1  int main() {
2      int x;
3      float y = 3.14
4  };
5
```

四、示例代码库和文法的解释

- 词法部分
- 示例程序10.src经过词法以后得到token序列

mytest.txt

```
1   (4,int)
2   (4,main)
3   (4,int)
4   (4,x)
5   (22,;)
6   (4,float)
7   (4,y)
8   (13,=)
9   (5,3)
10  (16,.)
11  (5,14)
12  (22,;)
13
```

E → i | f | d | d[E] | E + E | E * E | (E) | d(Ř)

- 表达式可以是i（整数），f（浮点数），d（某标识符）
- E+E，E*E，或者（），或者d（R）-函数调用表达式， 或者d（E）（这个文法，大作业不支持，也可以不写）

四、示例代码库和文法的解释

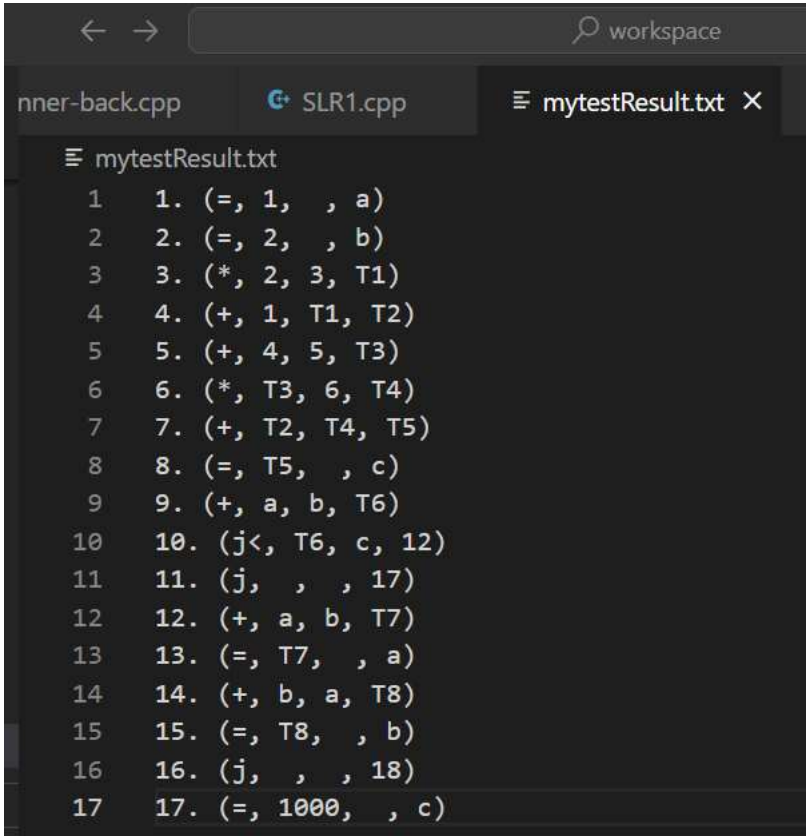
- 实验六输入

```
int main  ()
          a=1;
          b=2;
          c=1+2*3+(4+5)*6;

          if a+b<c

          {          a=a+b;
                     b=b+a
          }
          else
                     c=1000;
```

实验六输出



- 输入是一段符合文法的代码
- 输出是四元式方式
- 本次实验结果可作为验收结果